AI Agents for Beginners

Running the course notebooks on our internal LLM proxy

An intern onboarding guide - what to change in each notebook,
and the reasoning behind every change.

Internal training material - Lessons 01-08 covered

Model: gpt-5.1 | Endpoint: https://llmproxy.ustrnd.com (OpenAI-compatible, LiteLLM)

1. Why we need these changes

The course was written by Microsoft to run against Azure AI Foundry. Out of the box, every notebook expects an Azure subscription, an Azure AI Foundry project, and an `az login` session for credential-based authentication.

We do not use Azure. Instead, we have an internal, OpenAI-compatible LLM proxy (built on LiteLLM) that serves the gpt-5.1 model. So for each notebook we make a small, repeatable set of edits that point the code at our proxy and authenticate with an API key instead of Azure credentials.

This document lists those edits lesson by lesson, and - importantly - explains the reasoning so you understand WHY each change is made, not just what to type.

> **The mental model**
> Azure client + az login ==> OpenAI-compatible client + API key.
> Everything else in the notebooks (tools, agents, workflows, Pydantic models) stays the same -
> only the way we connect to the model changes.

2. One-time setup (do this before any notebook)

2.1 Create and activate a virtual environment

From inside the cloned repo folder (Windows PowerShell):

```
python -m venv venv
venv\Scripts\Activate.ps1
```

> **Why a venv?**
> It isolates the course's Python packages from your system Python so versions never clash.
> On PowerShell use Activate.ps1 - 'source venv/bin/activate' is Linux/Mac only.
> If you get a 'running scripts is disabled' error, run once:
> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

2.2 Install the packages

```
pip install agent-framework python-dotenv truststore pydantic semantic-kernel
```

- agent-framework - the Microsoft Agent Framework SDK the notebooks use.
- python-dotenv - loads our credentials from the .env file.
- truststore - makes Python trust the corporate certificate store (see 2.4).
- pydantic - structured output models (used from Lesson 03 on).
- semantic-kernel - only needed for the Semantic Kernel notebooks (e.g. 02-semantic-kernel).

2.3 Create the .env file in the repo root

Create a file named .env (note the leading dot) in the root of the repo and put in your proxy details:

```
FOUNDRY_PROJECT_ENDPOINT="https://llmproxy.ustrnd.com/v1"
FOUNDRY_API_KEY="sk-...your-key..."
FOUNDRY_MODEL_ID="gpt-5.1"
```

> **Important details about .env**
> Use .env, NOT .env.example. The example file is a committed template with placeholder values - never put a real key in it. .env is gitignored, so your key is never pushed to GitHub.
> The endpoint MUST end in /v1 - otherwise you get a 404 (the proxy serves the OpenAI API there).
> Ask the training team for your key. Treat it like a password.

2.4 Understand the corporate-network certificate issue

Our network performs TLS interception: it presents its own root certificate. Windows tools like curl trust it automatically (they use the Windows certificate store), but Python ships its own certificate list and does NOT trust the corporate root by default.

The symptom is a 'ConnectError / Connection error' even though the proxy is reachable in a browser or curl. The fix is the truststore package, which we add at the top of each notebook's first cell:

```
import truststore
truststore.inject_into_ssl()
```

This tells Python to validate certificates against the Windows store - the same one curl uses - so the TLS handshake succeeds.

2.5 Select the venv as the notebook kernel, then Restart + Run All

- Top-right of the notebook in VS Code, pick the Python interpreter from ...\venv\Scripts\python.exe.
- After editing any cell, always Restart the kernel and Run All - this clears any cached env var or client from a previous failed run.

3. The conversion recipe (applies to most lessons)

Almost every Agent Framework notebook needs the same four mechanical edits. Learn these once and you can convert any lesson.

Fix 1 - Swap the client class

Replace the Azure Foundry client with the OpenAI-compatible client.

```
# BEFORE (Azure):
from agent_framework.foundry import FoundryChatClient
from azure.identity import AzureCliCredential
client = FoundryChatClient(project_endpoint=..., model=...,
                           credential=AzureCliCredential())

# AFTER (our proxy):
from agent_framework.openai import OpenAIChatClient
client = OpenAIChatClient(base_url=endpoint, api_key=api_key, model=model)
```

Why: FoundryChatClient only does Azure credential auth - it does not even accept an api_key. OpenAIChatClient works with any OpenAI-compatible endpoint. Note the argument is model=, not model_id=.

Fix 2 - Read our proxy details from .env

```
from dotenv import load_dotenv
load_dotenv(override=True)
api_key  = os.getenv("FOUNDRY_API_KEY")
endpoint = os.getenv("FOUNDRY_PROJECT_ENDPOINT")
model    = os.getenv("FOUNDRY_MODEL_ID", "gpt-5.1")
```

Why: points the code at our values. override=True matters - without it an edited .env will NOT reload inside an already-running kernel, and you will keep using the old value.

Fix 3 - Add the TLS line at the very top of the imports cell

```
import truststore
truststore.inject_into_ssl()
```

Why: the corporate TLS-interception fix from section 2.4. Without it you get 'Connection error'.

Fix 4 - Replace provider.create_agent(...) with the Agent class

```
# BEFORE:  agent = await provider.create_agent(name=..., instructions=..., tools=[...])
# AFTER:
agent = Agent(client=client, name=..., instructions=..., tools=[...])
```

Why: 'provider' is the Azure pattern and is never defined in our setup (you get NameError: name 'provider' is not defined). Build the agent from the Agent class with client=client, and REMOVE the await - Agent(...) is not async; only agent.run(...) is. Some lessons have two or three of these per notebook - fix every one.

> **Golden rule**
> Every 'await provider.create_agent(' becomes 'Agent(client=client, ' (and drop the await).
> This single substitution is the most common edit across the whole course.

4. Per-notebook changes and reasoning

Lesson 01 - 01-python-agent-framework.ipynb

What it teaches: your first agent (a travel agent with one tool) + streaming.

- Applied Fixes 1-4 (the standard recipe).
- Client cell: FoundryChatClient -> OpenAIChatClient.
- Hit and resolved every environment issue here first: PowerShell activation, the TLS ConnectError (truststore), a 401 because the kernel cached an old key (load_dotenv(override=True) + restart), and a 404 fixed by adding /v1 to the endpoint.

> **Lesson 01 take-aways (apply everywhere)**
> Connection error -> truststore. 401 token_not_found -> wrong/cached key, use override=True + restart.
> 404 Not Found -> endpoint must end in /v1.

Lesson 02 - 02-python-agent-framework.ipynb

What it teaches: the four MAF building blocks - Client, Agent, Tools, Session.

- Applied the standard recipe (Fixes 1-4).
- Two agent cells used provider.create_agent -> converted to Agent(client=client, ...).
- Multi-turn session code (agent.create_session(), agent.run(..., session=session)) needed NO change - create_session() is synchronous and works with the OpenAI client.

Lesson 02 - 02-semantic-kernel.ipynb (DIFFERENT framework)

What it teaches: the same idea using Semantic Kernel instead of MAF.

This notebook already uses an OpenAI-compatible client (AsyncOpenAI) pointed at GitHub Models, so there is NO client-class swap. We just repoint it at our proxy.

```
client = AsyncOpenAI(
    api_key=os.environ.get("FOUNDRY_API_KEY"),
    base_url=os.environ.get("FOUNDRY_PROJECT_ENDPOINT"),
)
chat_completion_service = OpenAIChatCompletion(
    ai_model_id=os.environ.get("FOUNDRY_MODEL_ID", "gpt-5.1"),
    async_client=client,
)
```

> **Watch out**
> Semantic Kernel uses ai_model_id (not model). Do NOT change it to model=.
> Still add truststore at the top and pip install semantic-kernel.

Lesson 03 - 03-python-agent-framework.ipynb

What it teaches: agentic design patterns - clear instructions, structured output (Pydantic), single-responsibility agents.

- Standard recipe. The client is created at the top of the 'Pattern 1' cell - swap just that part to OpenAIChatClient.
- Patterns 2 and 3 each create agents via provider.create_agent -> converted to Agent(client=client, ...). Pattern 3 has TWO agents to convert.
- Fixed a small content bug: the logistics prompt used a plain string '{dest_response}' instead of an f-string, so the recommendation was never inserted. Added the f prefix.

Lesson 04 - 04-python-agent-framework.ipynb

What it teaches: the Tool Use pattern - multiple @tool functions, structured output, approval modes.

- Standard recipe. Client cell is separate and clean.
- Two agent cells (TravelToolAgent, StructuredTravelAgent) converted from provider.create_agent to Agent(client=client, ...).
- The @tool definitions and the always_require approval demo cell needed no change.

Lesson 05 - 05-python-agent-framework.ipynb

What it teaches: Agentic RAG - the agent decides when to retrieve, plus the maker-checker pattern.

- Despite the prerequisites mentioning Azure AI Search, the code uses an in-memory dictionary as the 'search' tool - so NO Azure Search keys are needed.
- Standard recipe. Two agent cells (TravelRAGAgent, TravelRAGCheckerAgent) converted to Agent(client=client, ...).

Lesson 06 - 06-system-message-framework.ipynb (DIFFERENT SDK)

What it teaches: building good system prompts.

This notebook uses the older Azure AI Inference SDK (azure.ai.inference.ChatCompletionsClient with client.complete(...)), not the Agent Framework. We rewrote it on the plain OpenAI SDK, which we know works with our proxy.

```
from openai import OpenAI
client = OpenAI(api_key=api_key, base_url=endpoint)
response = client.chat.completions.create(
    model=model_name,
    messages=[{"role": "system", "content": "..."},
              {"role": "user", "content": "..."}],
    temperature=1.0, max_tokens=1000,
)
print(response.choices[0].message.content)
```

> **New gotcha - unsupported parameters**
> Our proxy fronts Azure deployments, which reject some OpenAI sampling params.
> We removed top_p because the proxy returned: 'azure does not support parameters: [top_p]'.
> If you see '400 ... UnsupportedParamsError', just delete the named parameter from the call.

Lesson 07 - 07-python-agent-framework.ipynb

What it teaches: the Planning pattern - decompose a task into structured subtasks, then execute them.

- Standard recipe (Fixes 1-4), two agents converted (TravelPlanner, Concierge).

This lesson also exposed how structured output really works in the installed SDK. The original code read plan.destination directly off the response and crashed with: 'AgentResponse object has no attribute destination'. Two corrections:

- Request structured output via an options object, not a direct argument:

```
from agent_framework import ChatOptions
result = await planning_agent.run(
    "...",
    options=ChatOptions(response_format=TravelPlan),
)
plan = result.value     # the parsed Pydantic object
```

> **Structured output rule (Agent Framework)**
> Pass options=ChatOptions(response_format=YourModel) to run(...),
> then read the parsed object from response.value (NOT the response object itself).

Lesson 08 - 08-python-agent-framework.ipynb

What it teaches: multi-agent systems - several specialist agents wired into a sequential workflow.

This lesson genuinely uses the provider + workflow pattern, so there is slightly more to change:

- Replace 'provider = AzureAIProjectAgentProvider(credential=AzureCliCredential())' with 'client = OpenAIChatClient(...)'.
- Convert all three agents (TravelPlanner, TravelConcierge, BudgetReviewer) from provider.create_agent to Agent(client=client, ...).
- The WorkflowBuilder graph code (start_executor, add_edge, the streaming loop) needs NO change - it accepts Agent objects directly.

> **Provider + workflow conversion**
> AzureAIProjectAgentProvider(...) -> OpenAIChatClient(...)
> await provider.create_agent(...) -> Agent(client=client, ...)
> WorkflowBuilder code is unchanged.

5. Error -> fix cheat sheet

When a cell fails, match the error message to the fix:

`ConnectError / 'Connection error.'`	Corporate TLS interception. Add truststore.inject_into_ssl() at the top of the imports cell.
`401 token_not_found_in_db`	Key wrong or cached. Fix the key in .env, use load_dotenv(override=True), then Restart kernel + Run All.
`404 Not Found`	Endpoint is missing the API path. FOUNDRY_PROJECT_ENDPOINT must end in /v1.
`FoundryChatClient ... unexpected keyword 'api_key'`	Wrong client. Use OpenAIChatClient instead (Fix 1).
`OpenAIChatClient ... unexpected keyword 'model_id'`	Wrong arg name. Use model= not model_id=.
`name 'provider' is not defined`	Azure pattern. Use Agent(client=client, ...) and drop the await (Fix 4).
`Agent.run() got unexpected keyword 'response_format'`	Pass options=ChatOptions(response_format=Model) instead; read result.value.
`'AgentResponse' object has no attribute '...'`	You read a field off the wrapper. Use result.value after setting response_format.
`400 UnsupportedParamsError: azure does not support [param]`	Delete that parameter (e.g. top_p) from the create()/run() call.

6. Quick reference card

The .env file (repo root, gitignored)

```
FOUNDRY_PROJECT_ENDPOINT="https://llmproxy.ustrnd.com/v1"
FOUNDRY_API_KEY="sk-..."
FOUNDRY_MODEL_ID="gpt-5.1"
```

The standard imports + client block (Agent Framework lessons)

```
import truststore; truststore.inject_into_ssl()
import os
from dotenv import load_dotenv
from agent_framework import Agent, tool
from agent_framework.openai import OpenAIChatClient

load_dotenv(override=True)
api_key  = os.getenv("FOUNDRY_API_KEY")
endpoint = os.getenv("FOUNDRY_PROJECT_ENDPOINT")
model    = os.getenv("FOUNDRY_MODEL_ID", "gpt-5.1")

client = OpenAIChatClient(base_url=endpoint, api_key=api_key, model=model)
agent  = Agent(client=client, name='...', instructions='...', tools=[...])
```

> **Before you ask for help**
> 1. Is the venv activated and selected as the kernel?
> 2. Did you Restart kernel + Run All after editing?
> 3. Does .env end in /v1 and contain a valid key?
> 4. Is truststore.inject_into_ssl() at the very top?
>
> Most issues are one of these four.

Internal document for training use. Covers Lessons 01-08. Keep API keys out of git and out of shared documents - rotate any key that gets exposed.